

Optimisation Numérique et Sciences des Données

Optimisation numérique – Méthodes Itératives

Paul Catala (CRAN, UL)
paul.catala(at)univ-lorraine.fr

Master Ingénierie des Systèmes Complexes M2 ISC 925
Semestre 9 – 2024

Cours élaboré par Julien Flamant

- On veut résoudre un problème d'optimisation (continue) sans contraintes

$$\min_{x \in \mathbb{R}^n} f(x)$$

Dans la très grande majorité des cas, il n'y a **pas de solution analytique** comme pour les moindres carrés

- On veut résoudre un problème d'optimisation (continue) sans contraintes

$$\min_{x \in \mathbb{R}^n} f(x)$$

Dans la très grande majorité des cas, il n'y a **pas de solution analytique** comme pour les moindres carrés

- **Stratégie:** **méthodes itératives** (algorithmes)
 1. on **choisit** un (ou plusieurs) point initial $x_0 \in \mathbb{R}^n$
 2. à chaque itération k , on obtient un nouveau point x_k suivant une mise à jour construite à partir de la fonction f et de l'itérée x_{k-1}
 3. on réitère jusqu'à ce qu'un critère d'arrêt soit satisfait

Algorithme d'optimisation sans gradient

→ il suffit de pouvoir évaluer la fonction $f(\mathbf{x})$

✓ Méthode heuristique bien pratique lorsque f n'est pas différentiable

✗ pas de garantie de convergence vers un minimiseur local

Idée générale pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$

1. On tire aléatoirement $n + 1$ points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n+1}$ dans $\mathbb{R}^n$

Algorithme d'optimisation sans gradient

→ il suffit de pouvoir évaluer la fonction $f(\mathbf{x})$

✓ Méthode heuristique bien pratique lorsque f n'est pas différentiable

✗ pas de garantie de convergence vers un minimiseur local

Idée générale pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$

1. On tire aléatoirement $n + 1$ points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n+1}$ dans $\mathbb{R}^n$
2. on construit le simplexe associé (polytope convexe à $n + 1$ sommets : un triangle en 2D, un tétraèdre en 3D...)

Algorithme d'optimisation sans gradient

→ il suffit de pouvoir évaluer la fonction $f(\mathbf{x})$

✓ Méthode heuristique bien pratique lorsque f n'est pas différentiable

✗ pas de garantie de convergence vers un minimiseur local

Idée générale pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$

1. On tire aléatoirement $n + 1$ points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n+1}$ dans $\mathbb{R}^n$
2. on construit le simplexe associé (polytope convexe à $n + 1$ sommets : un triangle en 2D, un tétraèdre en 3D...)
3. on évalue f en chaque sommet

Algorithme d'optimisation sans gradient

→ il suffit de pouvoir évaluer la fonction $f(\mathbf{x})$

✓ Méthode heuristique bien pratique lorsque f n'est pas différentiable

✗ pas de garantie de convergence vers un minimiseur local

Idée générale pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$

1. On tire aléatoirement $n + 1$ points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n+1}$ dans $\mathbb{R}^n$
2. on construit le simplexe associé (polytope convexe à $n + 1$ sommets : un triangle en 2D, un tétraèdre en 3D...)
3. on évalue f en chaque sommet
4. on en déduit une transformation de ce simplexe pour le faire « ramper » vers un minimiseur (local) à l'aide d'opérations de :
réflexion, expansion, contraction, rétrécissement

Algorithme d'optimisation sans gradient

→ il suffit de pouvoir évaluer la fonction $f(\mathbf{x})$

✓ Méthode heuristique bien pratique lorsque f n'est pas différentiable

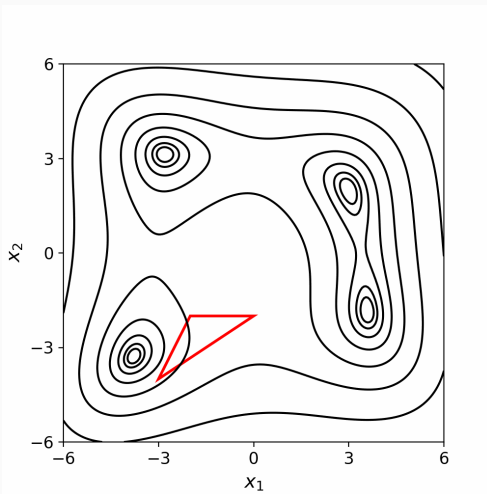
✗ pas de garantie de convergence vers un minimiseur local

Idée générale pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$

1. On tire aléatoirement $n + 1$ points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n+1}$ dans $\mathbb{R}^n$
2. on construit le simplexe associé (polytope convexe à $n + 1$ sommets : un triangle en 2D, un tétraèdre en 3D...)
3. on évalue f en chaque sommet
4. on en déduit une transformation de ce simplexe pour le faire « ramper » vers un minimiseur (local) à l'aide d'opérations de : réflexion, expansion, contraction, rétrécissement
5. on répète l'opération jusqu'à ce que le simplexe atteigne une taille suffisamment petite

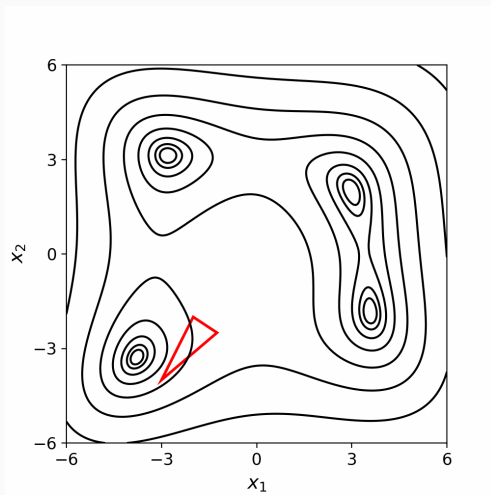
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



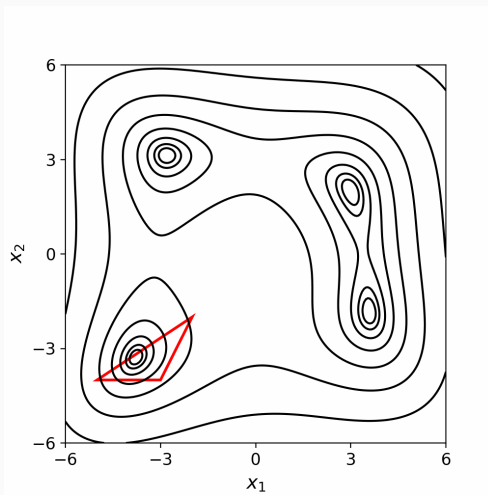
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



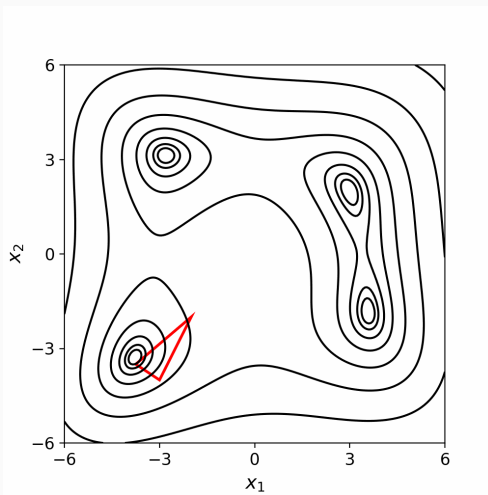
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



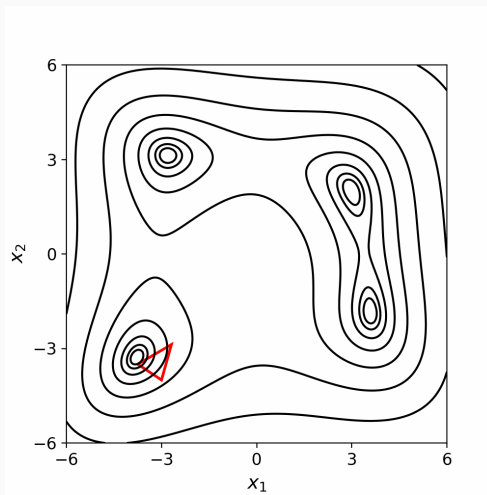
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



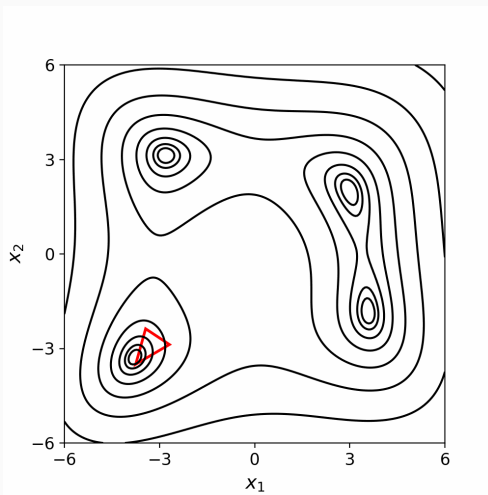
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



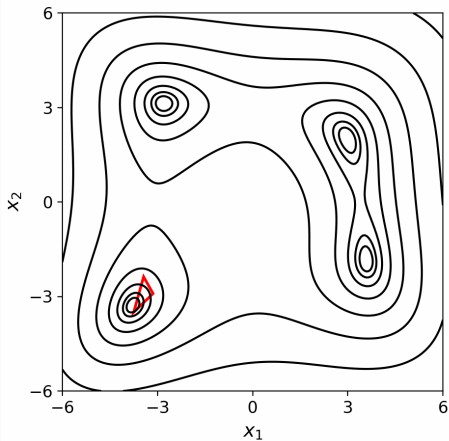
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



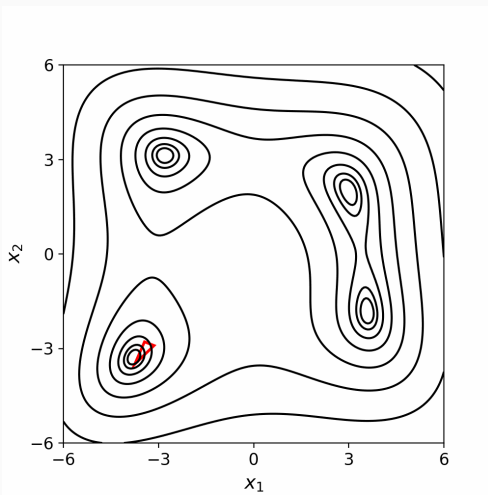
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



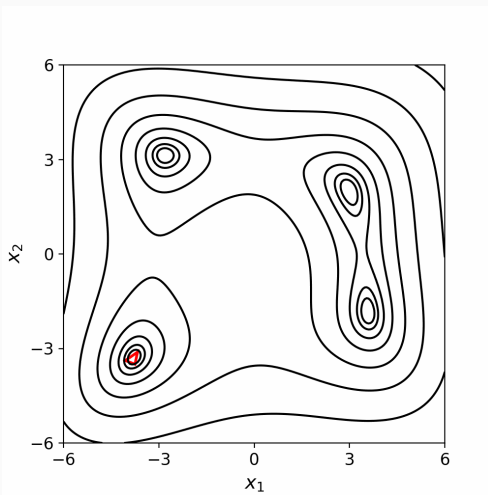
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



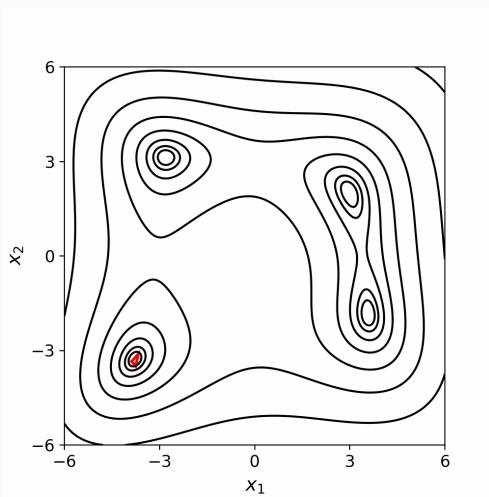
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



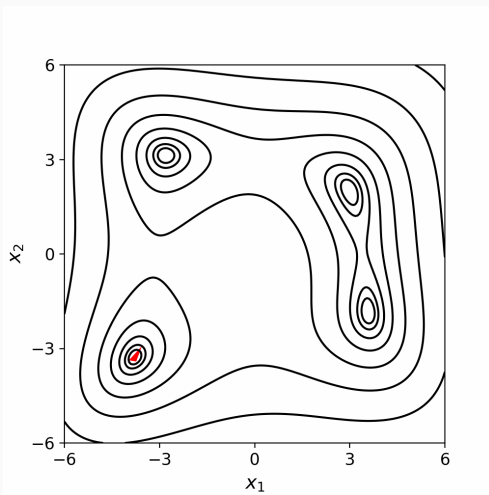
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



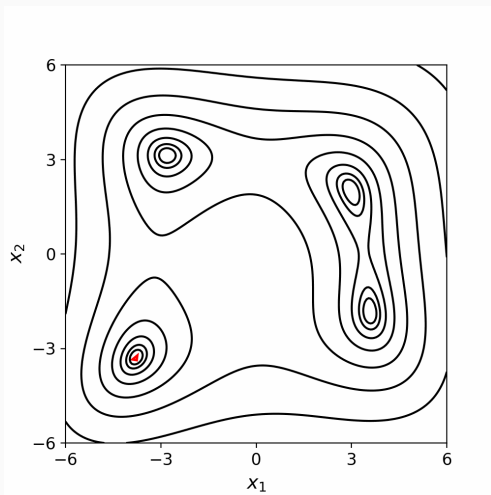
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



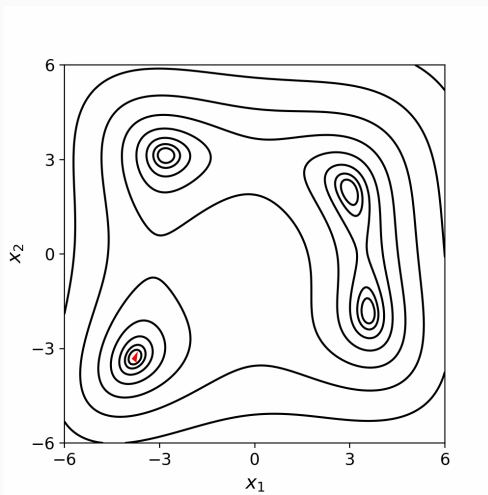
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



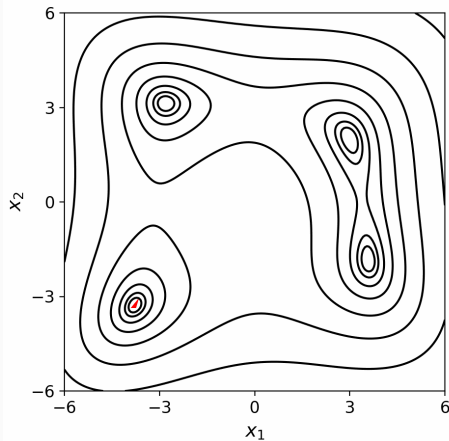
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



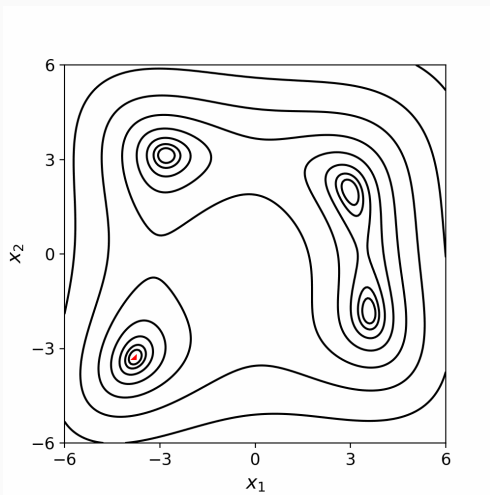
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



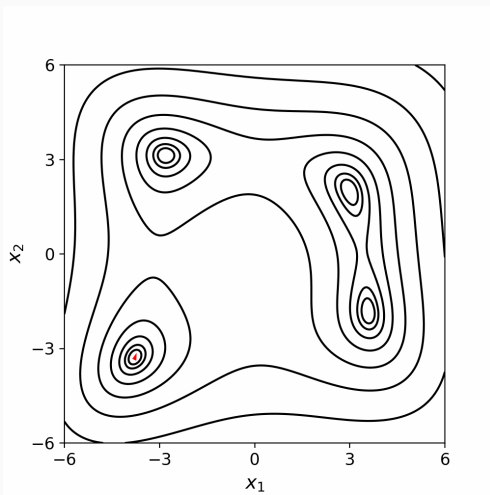
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



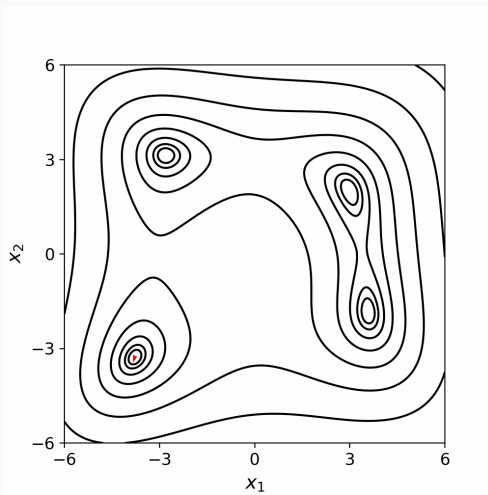
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



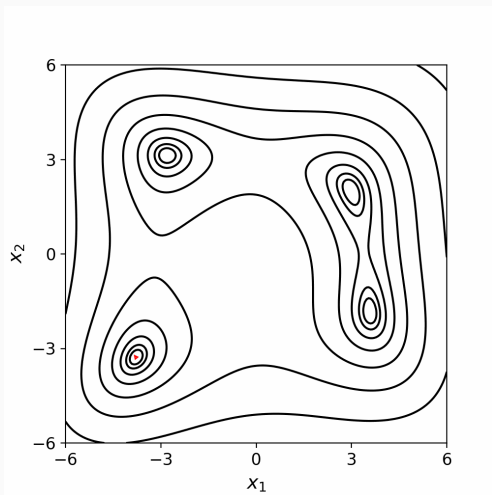
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



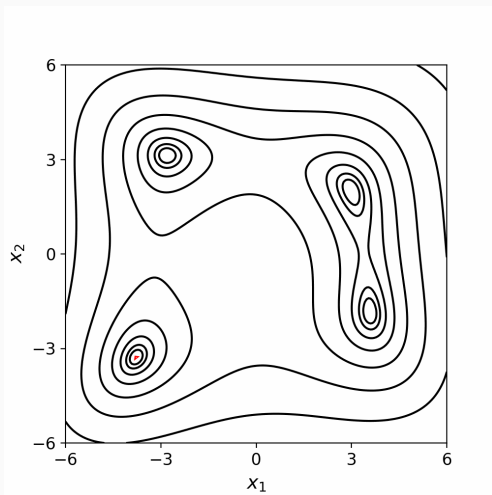
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



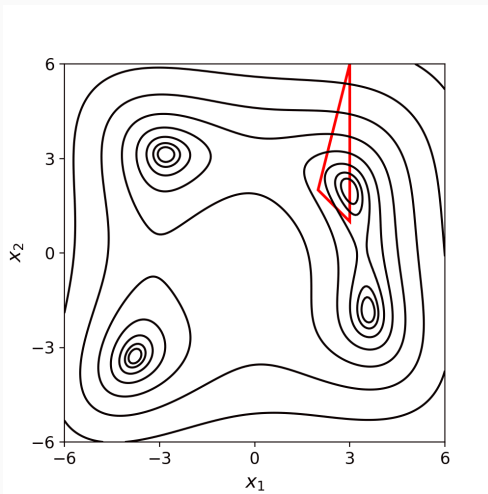
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



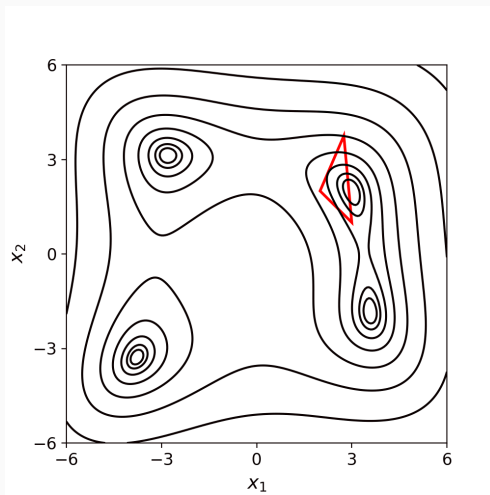
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



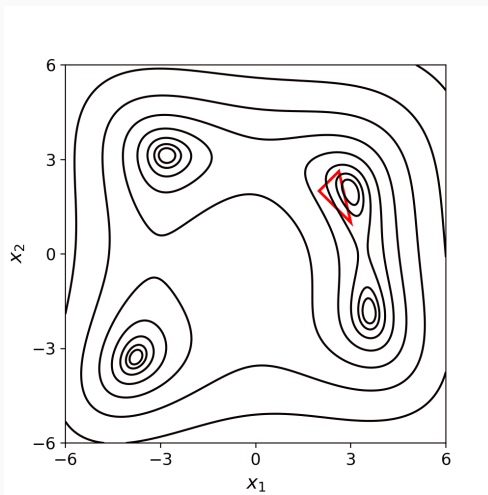
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



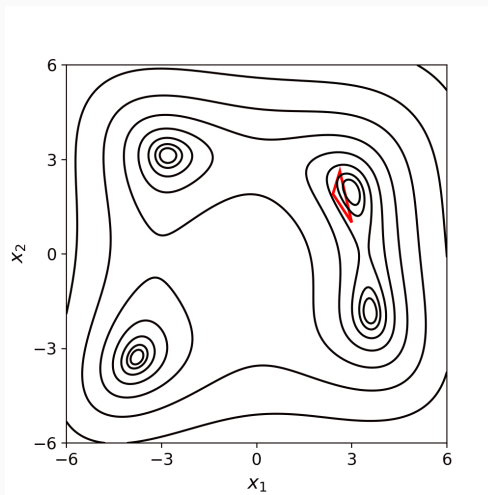
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



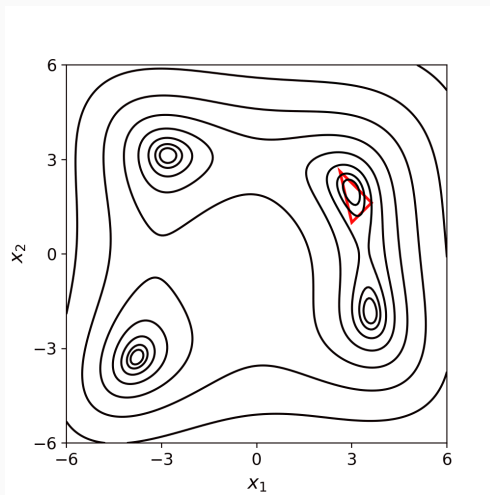
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



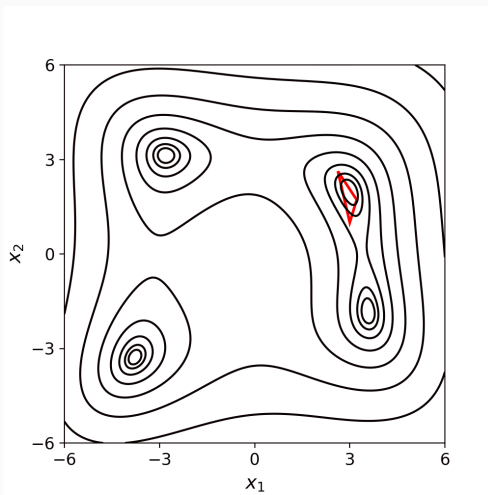
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



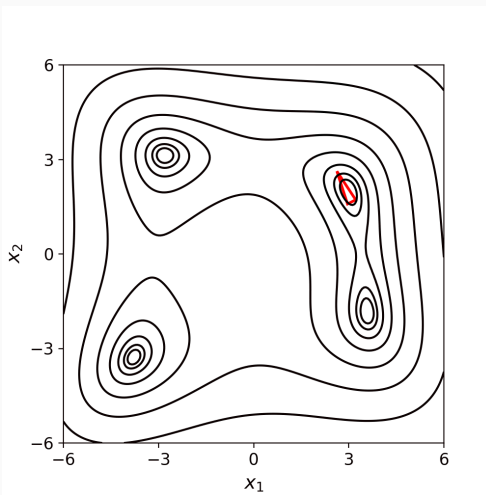
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



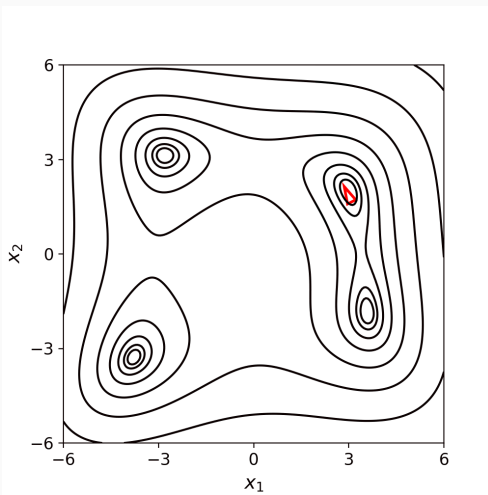
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



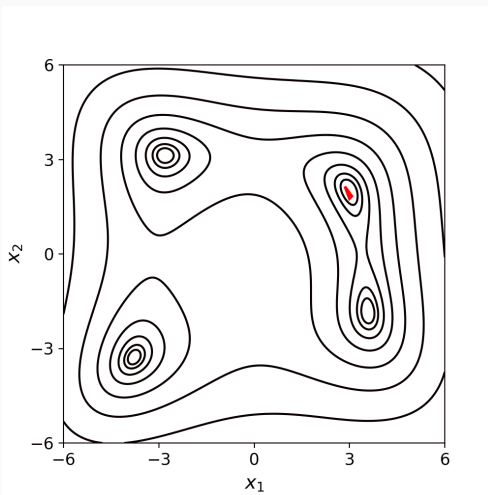
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



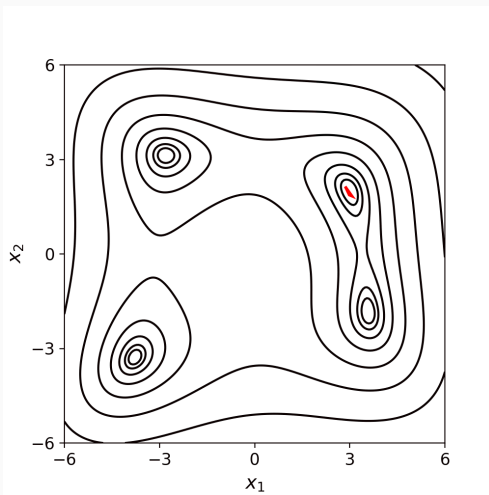
EXAMPLE

L'algorithme peut converger vers un minimiseur local ...



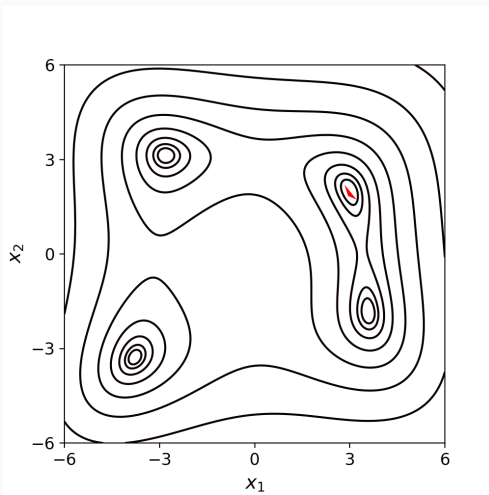
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



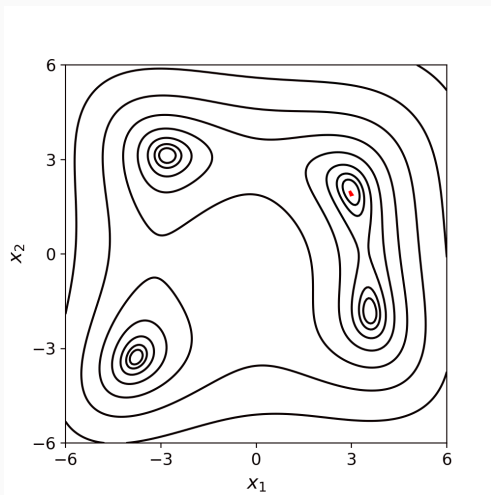
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



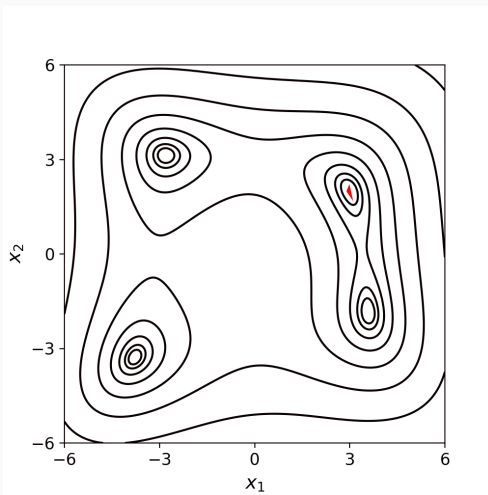
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



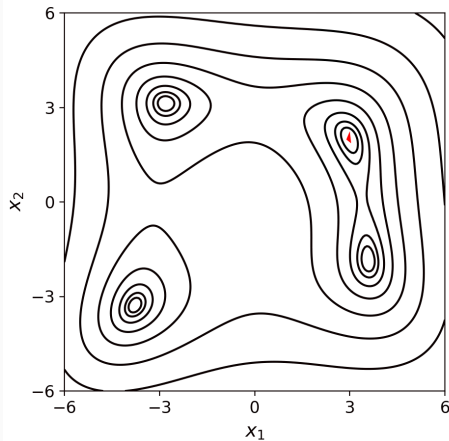
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



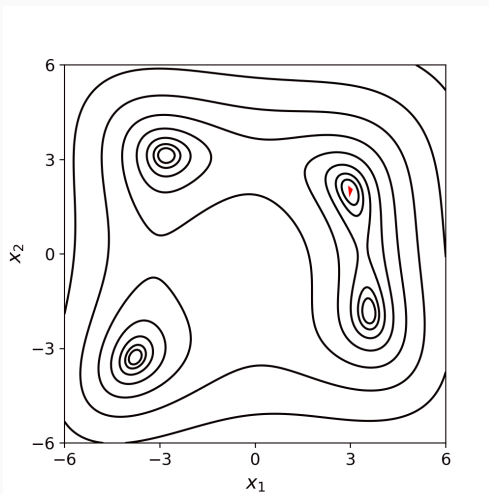
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



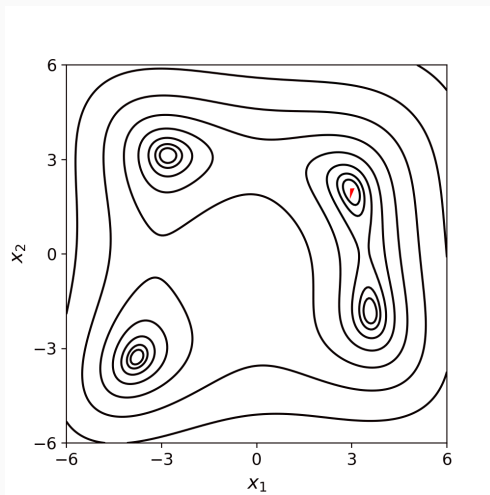
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



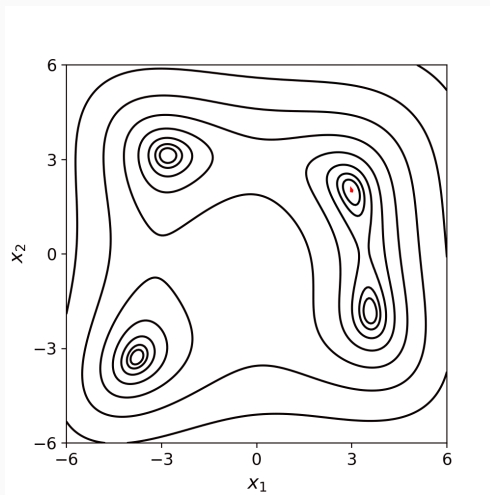
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



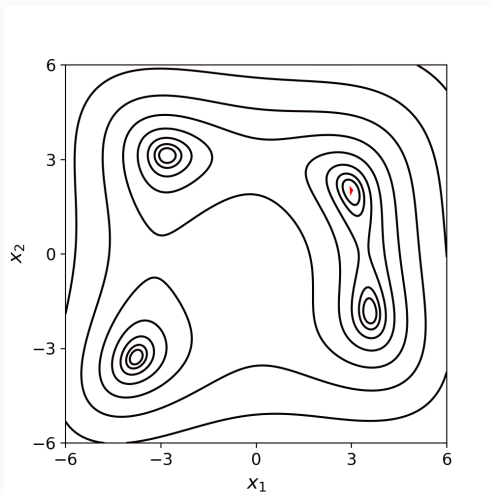
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



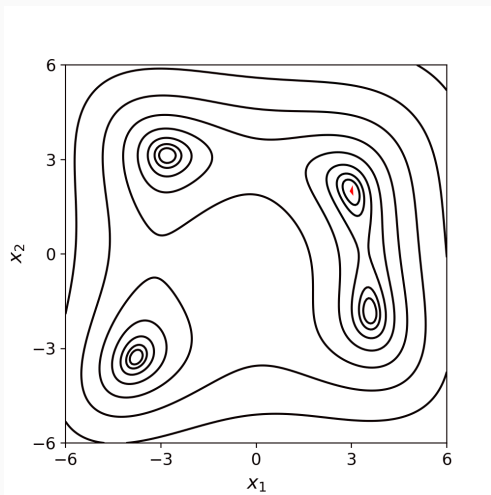
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



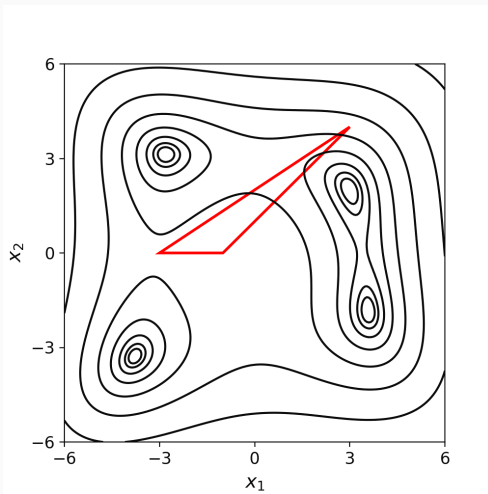
EXEMPLE

L'algorithme peut converger vers un minimiseur local ...



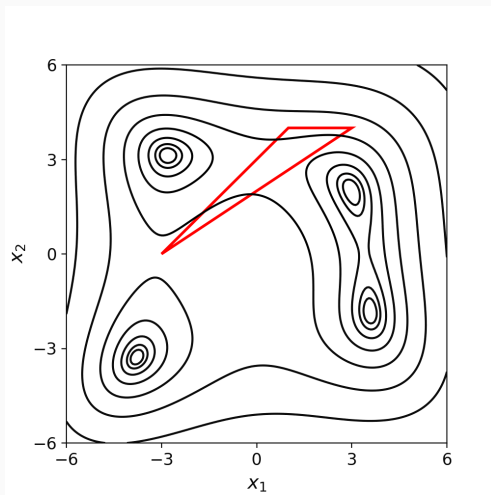
EXEMPLE

... mais pas toujours



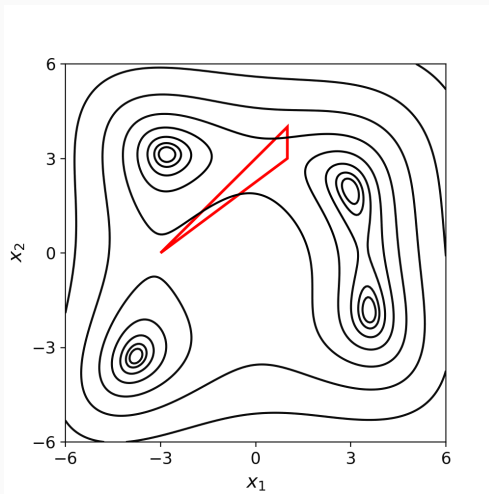
EXEMPLE

... mais pas toujours



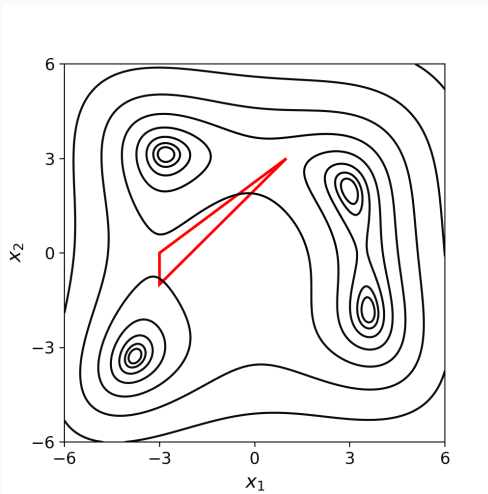
EXEMPLE

... mais pas toujours



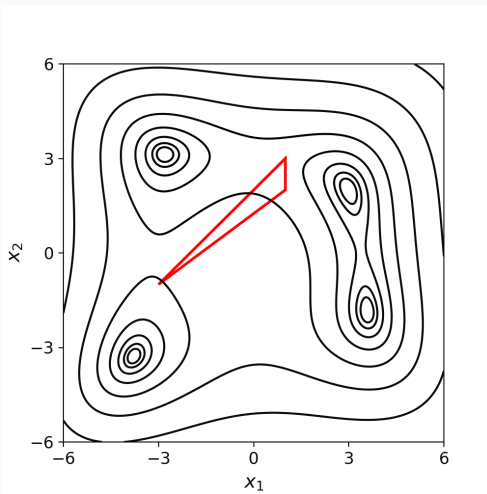
EXEMPLE

... mais pas toujours



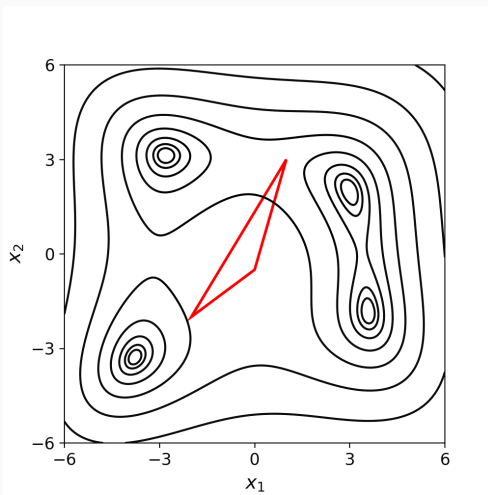
EXAMPLE

... mais pas toujours



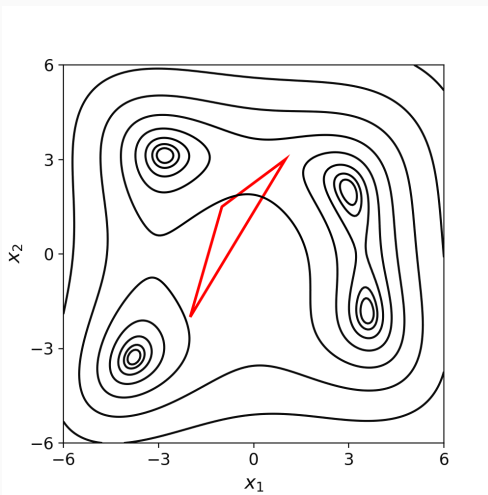
EXAMPLE

... mais pas toujours



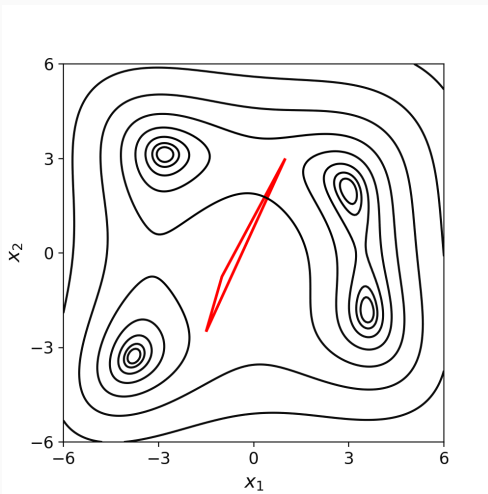
EXEMPLE

... mais pas toujours



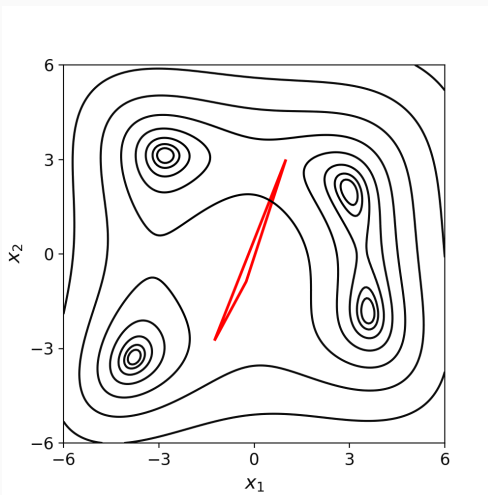
EXEMPLE

... mais pas toujours



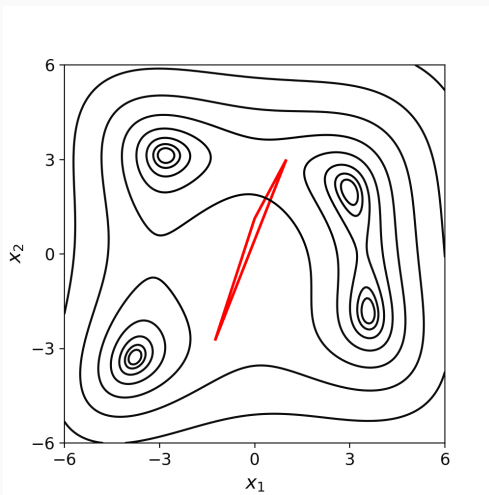
EXEMPLE

... mais pas toujours



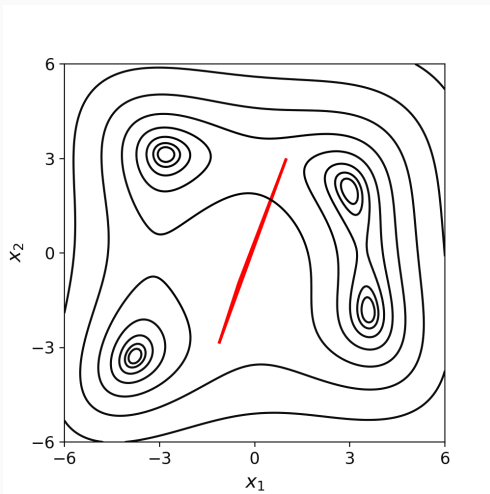
EXEMPLE

... mais pas toujours



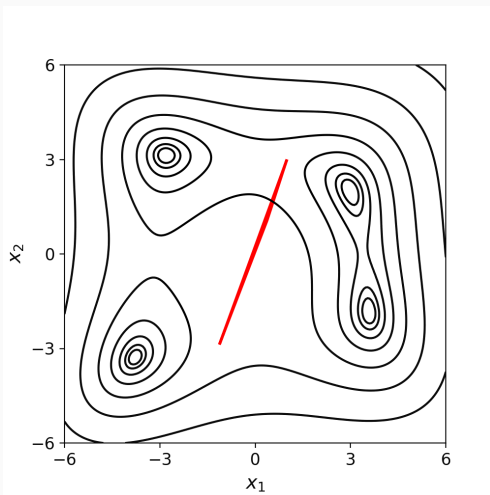
EXEMPLE

... mais pas toujours



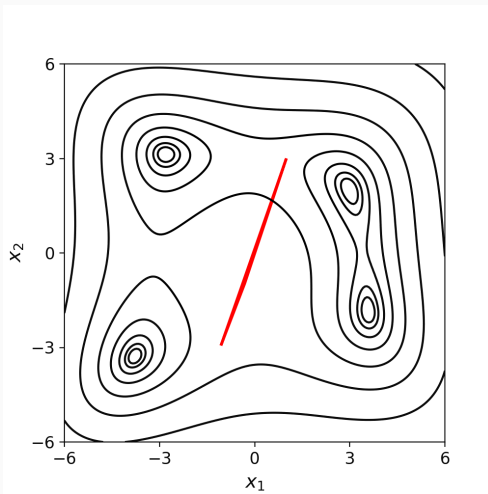
EXEMPLE

... mais pas toujours



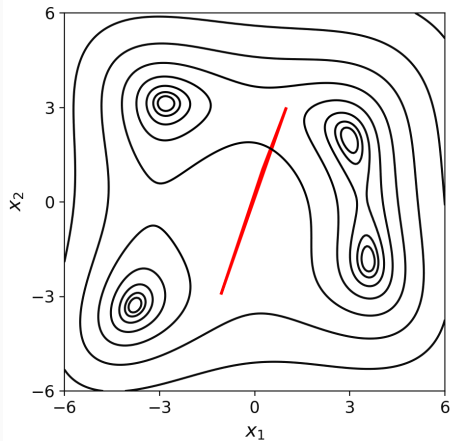
EXEMPLE

... mais pas toujours



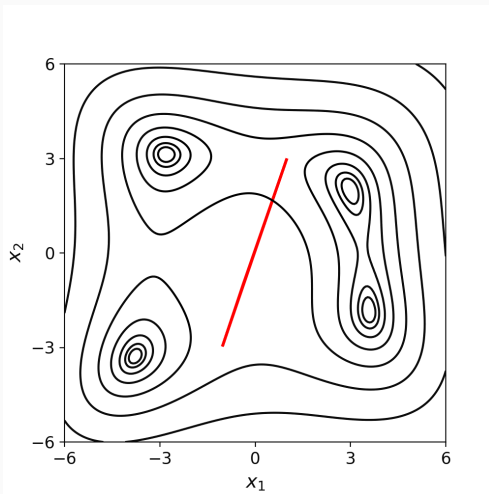
EXEMPLE

... mais pas toujours



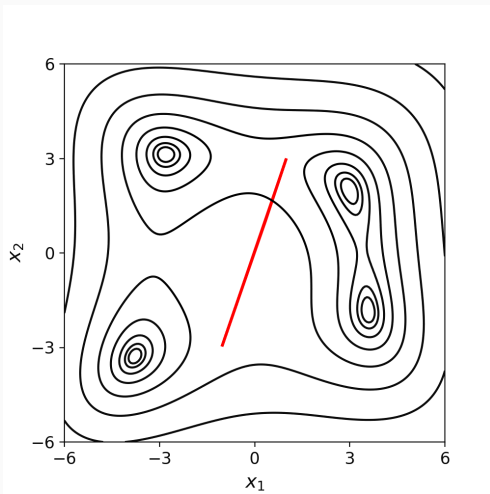
EXEMPLE

... mais pas toujours



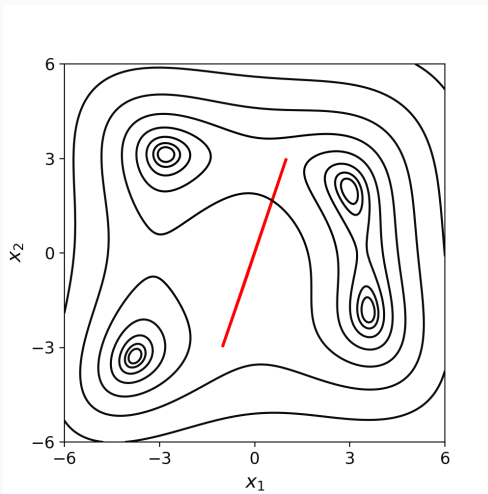
EXEMPLE

... mais pas toujours



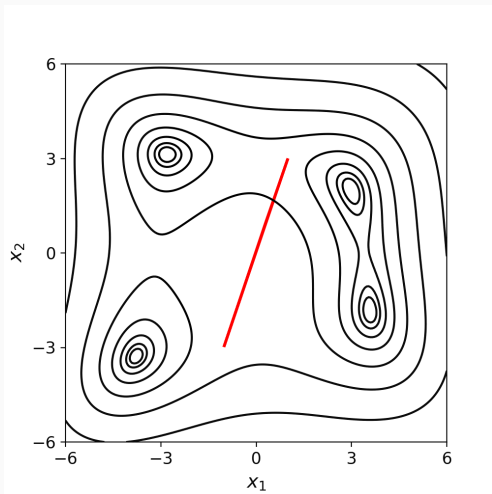
EXEMPLE

... mais pas toujours



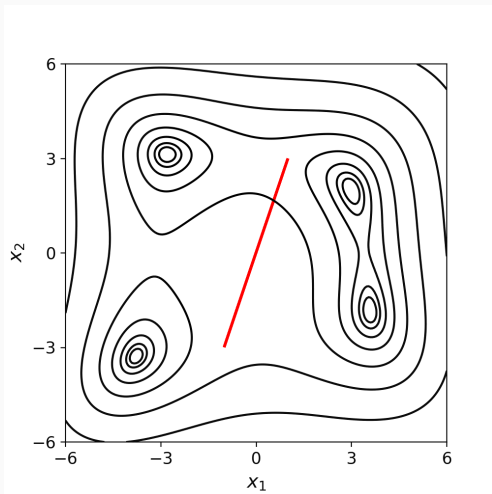
EXEMPLE

... mais pas toujours



EXEMPLE

... mais pas toujours



- A l'itération k , on met à jour

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

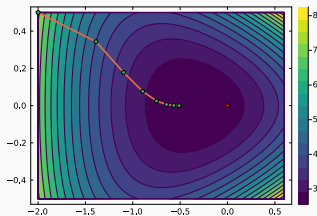
où $\alpha_k \geq 0$ est le **pas** et $\mathbf{d}_k$ une **direction**.

- A l'itération k , on met à jour

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

où $\alpha_k \geq 0$ est le **pas** et $\mathbf{d}_k$ une **direction**.

- **Objectif rêvé**: on veut $\mathbf{x}_k \rightarrow \mathbf{x}_*$ avec $\mathbf{x}_*$ un **minimiseur global** de f

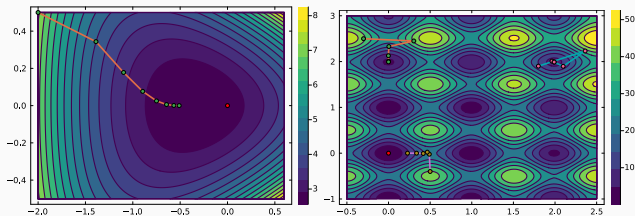


- A l'itération k , on met à jour

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

où $\alpha_k \geq 0$ est le **pas** et $\mathbf{d}_k$ une **direction**.

- **Objectif rêvé**: on veut $\mathbf{x}_k \rightarrow \mathbf{x}_*$ avec $\mathbf{x}_*$ un **minimiseur global** de f



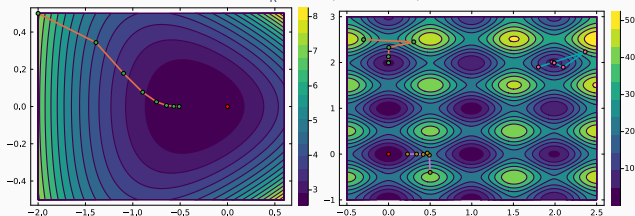
→ Rarement possible car la fonction f n'est pas toujours **convexe**

- A l'itération k , on met à jour

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

où $\alpha_k \geq 0$ est le **pas** et $\mathbf{d}_k$ une **direction**.

- **Objectif réaliste**: on construit $\mathbf{x}_k \rightarrow \mathbf{x}_*$ avec $\mathbf{x}_*$ un **minimiseur local** de f



On veut

- que la suite $\{\mathbf{x}_k\}$ décrive un **algorithme de descente**, i.e.

$$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq \dots \geq f(\mathbf{x}_k) \geq f(\mathbf{x}_{k+1}) \geq \dots$$

→ Pour garantir cela, on choisit adéquatement la direction et le pas

- que l'algorithme s'arrête lorsqu'il ne progresse plus

→ on définit un **critère d'arrêt**.

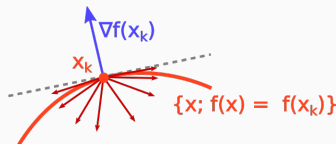
- On dit que $\mathbf{d}$ est une direction de descente à partir d'un point $\mathbf{x} \in \mathbb{R}^n$ lorsque

$$\exists \alpha > 0 \quad \text{t.q.} \quad f(\mathbf{x}) \geq f(\mathbf{x} + \alpha \mathbf{d})$$

- Condition de descente au point $\mathbf{x}_k$

$$\nabla f(\mathbf{x}_k) \cdot \mathbf{d} < 0$$

Ce sont toutes les directions qui nous font "traverser" la ligne de niveau



NB: Intuition : développement de Taylor

$$f(\mathbf{x} + \alpha \mathbf{d}) = f(\mathbf{x}) + \alpha \nabla f(\mathbf{x}) \cdot \mathbf{d} + o(\|\mathbf{d}\|)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

Chaque choix la direction de descente donne un algorithme de descente différent

Forme générale:

$$\mathbf{d}_k = -\mathbf{B}_k^{-1} \nabla f(\mathbf{x}_k) \quad \text{avec} \quad \mathbf{B}_k \text{ symétrique et inversible}$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$$

Chaque choix la direction de descente donne un algorithme de descente différent

Forme générale:

$$\mathbf{d}_k = -\mathbf{B}_k^{-1} \nabla f(\mathbf{x}_k) \quad \text{avec} \quad \mathbf{B}_k \text{ symétrique et inversible}$$

Par exemple:

- Méthode de descente du gradient: $\mathbf{B}_k = \mathbf{I}_n$ (matrice identité)
- Méthode de Newton: $\mathbf{B}_k = \nabla^2 f(\mathbf{x}_k)$ (matrice Hessienne de f)
- Méthode de quasi-Newton: $\mathbf{B}_k \simeq \nabla^2 f(\mathbf{x}_k)$ (approx. Hessienne)

COMMENT DÉFINIR UN CRITÈRE D'ARRÊT?

- En théorie, l'algorithme renvoie une suite $\{x_k\}_{k=1,2,\dots}$ telle que

$$\lim_{k \rightarrow \infty} x_k = \text{un minimum local}$$

En pratique, il faut garantir qu'il s'arrête après un nombre fini d'itérations

- Idée: stopper l'algorithme lorsque

$$(\text{valeur du critère à l'itération } k) \leq \varepsilon$$

où ε est une (petite) tolérance / précision fixée (par ex $\varepsilon = 10^{-6}$)

COMMENT DÉFINIR UN CRITÈRE D'ARRÊT?

- En théorie, l'algorithme renvoie une suite $\{x_k\}_{k=1,2,\dots}$ telle que

$$\lim_{k \rightarrow \infty} x_k = \text{un minimum local}$$

En pratique, il faut garantir qu'il s'arrête après un nombre fini d'itérations

- Idée: stopper l'algorithme lorsque

$$(\text{valeur du critère à l'itération } k) \leq \varepsilon$$

où ε est une (petite) tolérance / précision fixée (par ex $\varepsilon = 10^{-6}$)

- Exemples de critères:

- variation de l'objectif: $|f(x_{k+1}) - f(x_k)| \leq \varepsilon$
- variation de l'itérée: $\|x_{k+1} - x_k\| \leq \varepsilon$
- norme du gradient (condition d'optimalité): $\|\nabla f(x_k)\| \leq \varepsilon$

et généralement une combinaison de ces critères

- On utilise de préférence des critères en valeur relative, e.g.

$$\frac{|f(x_{k+1}) - f(x_k)|}{|f(x_k)|} \quad \text{ou} \quad \frac{\|x_{k+1} - x_k\|}{\|x_k\|}$$

A partir d'un point initial $x_0 \in \mathbb{R}^n$, itérer:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Si le pas α_k est bien choisi, x_k converge vers un point stationnaire de f (i.e. $\nabla f(x_k) = 0$)

Stratégies de pas:

- pas constant: on fixe $\alpha_k = \alpha$ pour tout k
- pas optimal: à chaque itération k on résout

$$\alpha_k = \underset{\alpha \geq 0}{\operatorname{argmin}} f(x_k - \alpha \nabla f(x_k))$$

- méthode de recherche de ligne ou **backtracking**

Descente de gradient à pas constant

A partir d'un point initial x_0 , itérer

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

Intuitivement

- α trop petit: convergence lente (en terme du nombre d'itérations)
- α trop grand: **risque de divergence**

→ Le pas doit être choisi de manière à équilibrer entre vitesse de convergence et garanties de convergence

Descente de gradient à pas constant

A partir d'un point initial x_0 , itérer

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

Theorem (Convergence dans le cas Lipschitz).

Si f a un gradient L -Lipschitz, i.e. si

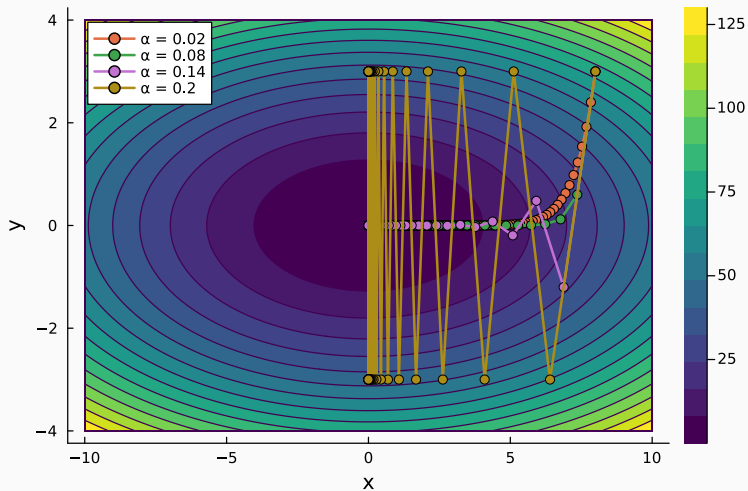
$$\forall x, y \in \mathbb{R}^n, \quad \|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$$

et si

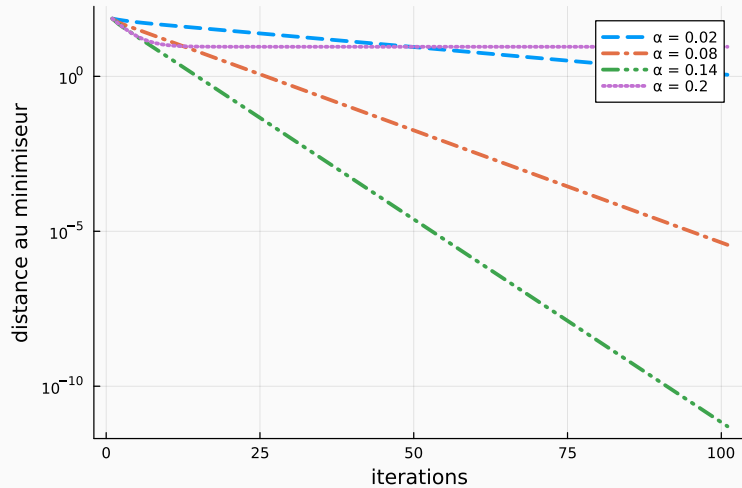
$$0 < \alpha < \frac{L}{2}$$

alors l'algorithme converge vers un point stationnaire de f .

UN EXEMPLE EN 2D



UN EXEMPLE EN 2D



Descente de gradient à pas optimal

A partir d'un point initial $x_0 \in \mathbb{R}^n$, itérer

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

où α_k est obtenu en résolvant le problème de minimisation

$$\alpha_k = \operatorname{argmin}_{\alpha \geq 0} f(x_k - \alpha \nabla f(x_k))$$

à chaque itération.

Descente de gradient à pas optimal

A partir d'un point initial $x_0 \in \mathbb{R}^n$, itérer

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

où α_k est obtenu en résolvant le problème de minimisation

$$\alpha_k = \operatorname{argmin}_{\alpha \geq 0} f(x_k - \alpha \nabla f(x_k))$$

à chaque itération.

Stratégie optimale en théorie

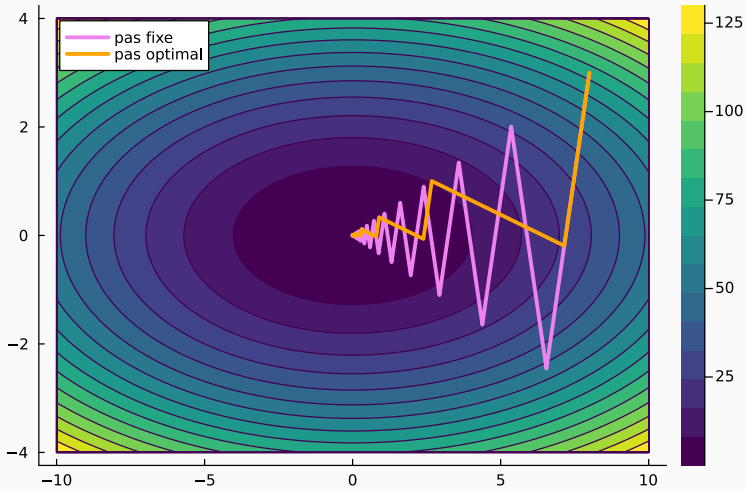
En pratique, calculer le pas optimal peut se révéler aussi difficile que le problème original, ou être très coûteux numériquement

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top Q \mathbf{x}, \quad \text{avec} \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & \gamma \end{bmatrix}, \gamma > 0$$

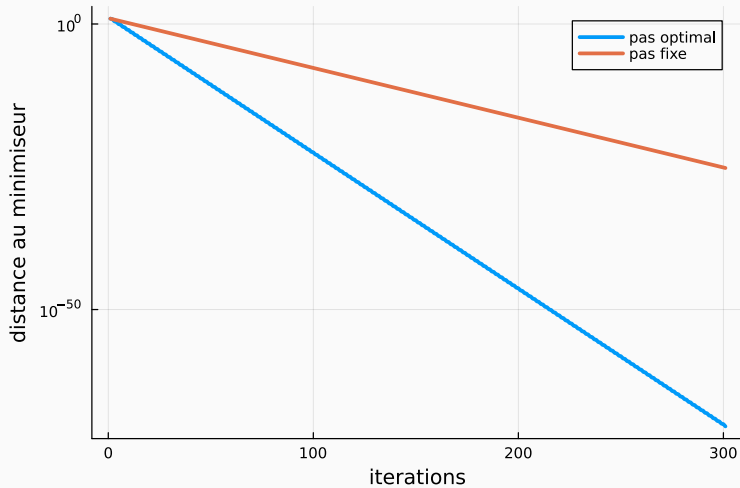
- Calculer $\nabla f(\mathbf{x})$
- Exprimer $\mathbf{x}_{k+1}$ en fonction de $\mathbf{x}_k$
- Exprimer $\mathbf{x}_1$ en fonction de α_0 si $\mathbf{x}_0 = [\gamma, 1]^\top$
- Calculer le pas optimal pour cette itération
- Montrer que les itérations avec pas optimal sont données par

$$\mathbf{x}_k = \gamma \left(\frac{\gamma - 1}{\gamma + 1} \right)^k \begin{bmatrix} 1 \\ (-1)^k \end{bmatrix}$$

EXEMPLE POUR $\gamma = 10$



EXEMPLE POUR $\gamma = 10$



- On cherche un pas α_k qui minimise suffisamment la fonction objectif f

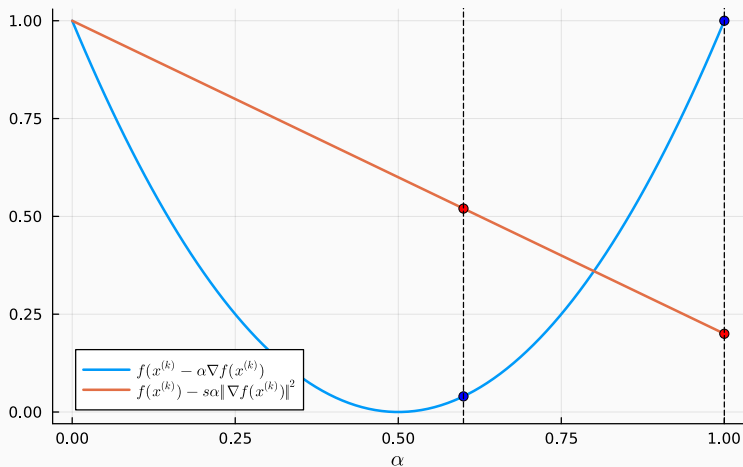
$$\alpha_k \simeq \operatorname{argmin}_{\alpha \geq 0} f(\mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k))$$

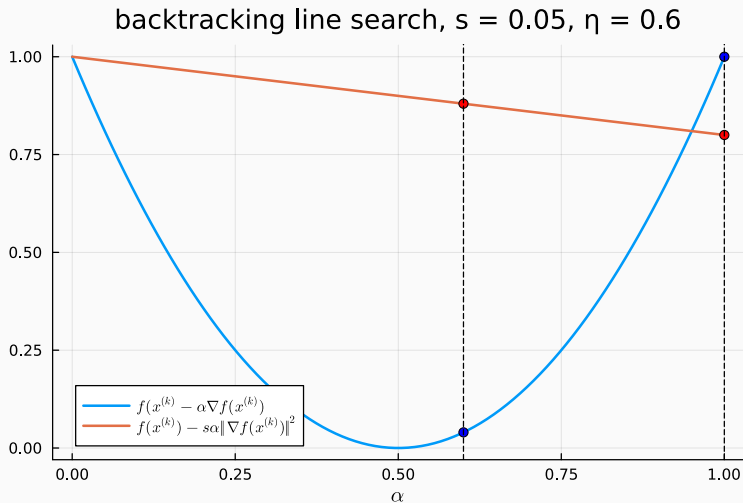
- Approche la plus utilisée en pratique: on réduit α ("backtracking") jusqu'à ce qu'un certain critère soit vérifié (e.g. condition d'Armijo, condition de Wolfe, ...)
- **Exemple** Armijo dans le cas convexe: on fixe $s \in (0, 0.5)$ et $\eta \in (0, 1)$ et un pas initial $\alpha = 1$

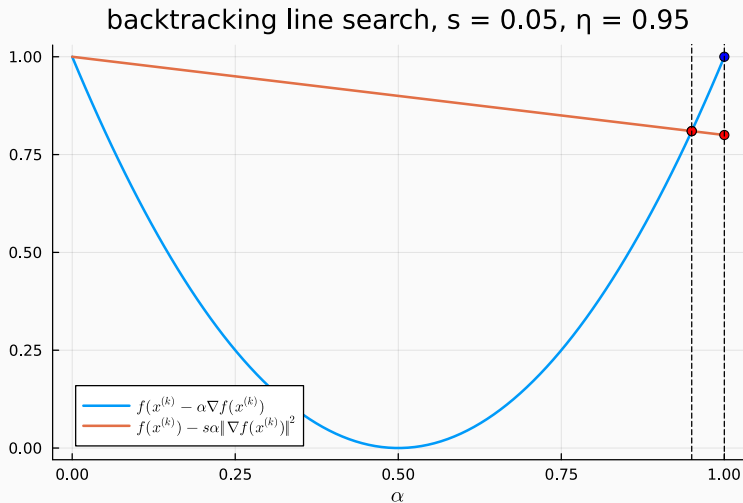
```
 $\alpha := 1$   
while  $f(\mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)) > f(\mathbf{x}_k) - \alpha s \|\nabla f(\mathbf{x}_k)\|^2$  do  
     $\alpha = \eta \alpha$   
end while
```

- Les stratégies approchées du choix de pas sont construites de manière à assurer la convergence de l'algorithme

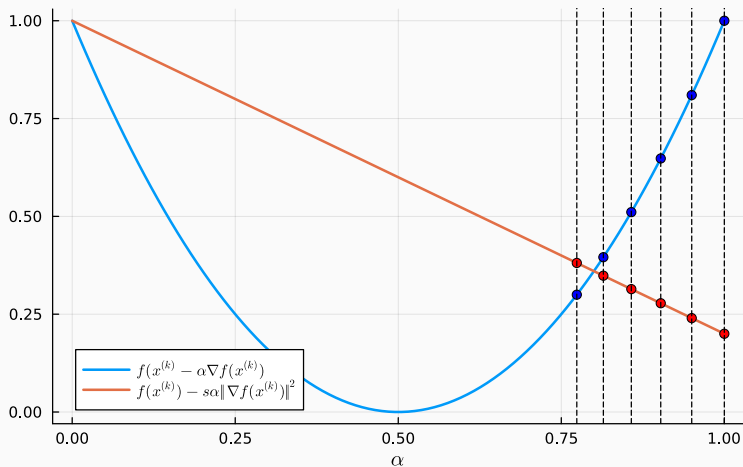
backtracking line search, $s = 0.2$, $\eta = 0.6$



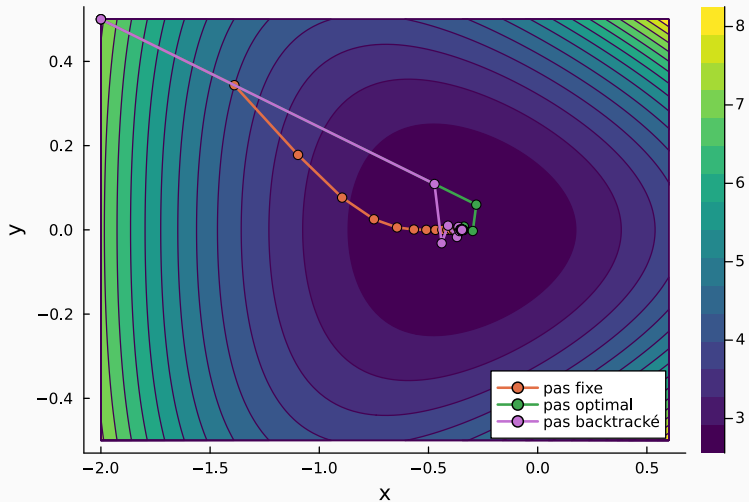




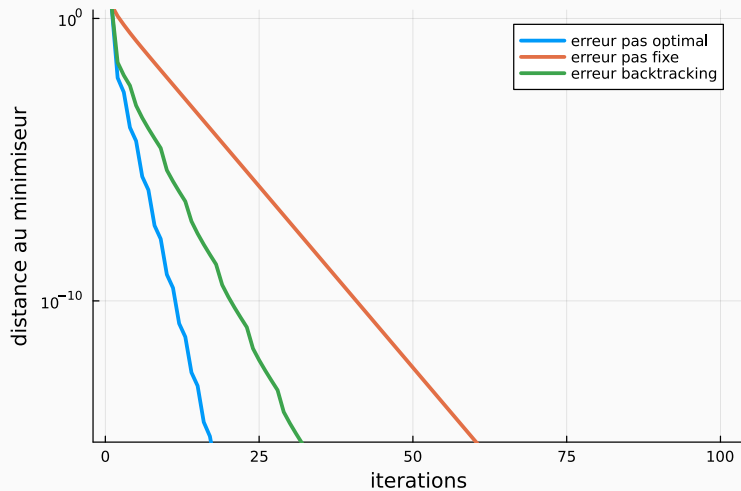
backtracking line search, $s = 0.2$, $\eta = 0.95$



COMPARAISON DES STRATÉGIES DE PAS



COMPARAISON DES STRATÉGIES DE PAS



A partir d'un point initial $x_0 \in \mathbb{R}^n$, itérer:

$$x_{k+1} = x_k - \alpha_k \left[\nabla^2 f(x_k) \right]^{-1} \nabla f(x_k)$$

$d_k := - \left[\nabla^2 f(x_k) \right]^{-1} \nabla f(x_k)$ est bien une direction de descente dès lors que la Hessienne de f en x_k est définie positive

$$\nabla f(x_k) \cdot d_k = -\nabla f(x_k) \left[\nabla^2 f(x_k) \right]^{-1} \nabla f(x_k) < 0$$

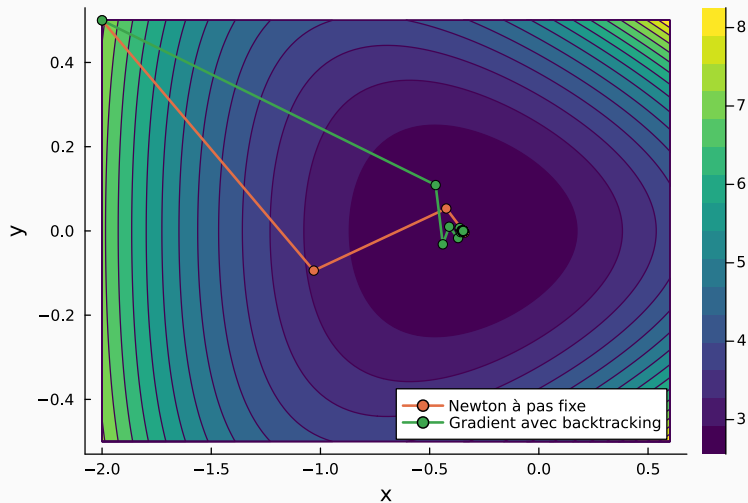
Stratégies de choix du pas

- pas constant: on fixe $\alpha_k = \alpha$ pour tout k (souvent $\alpha = 1$)
- utilisation du backtracking possible

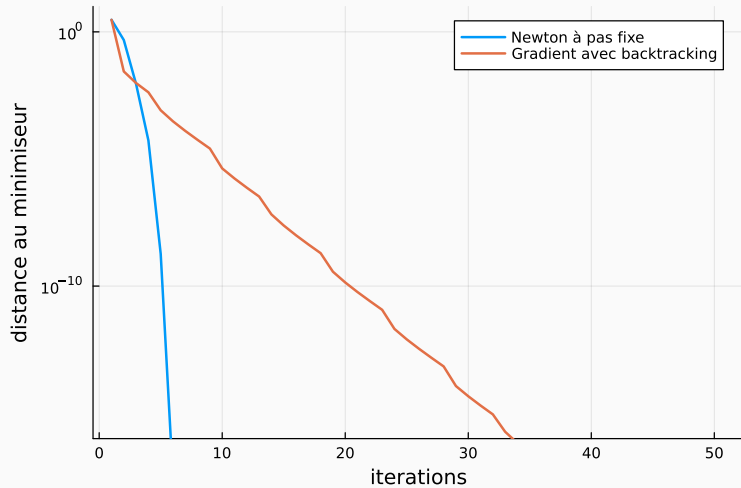
$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} - \mathbf{b}^\top \mathbf{x}, \quad \text{avec } \mathbf{Q} \succ 0$$

- Calculer $\nabla f(\mathbf{x})$ et en déduire le minimiseur de f
- Calculer $\nabla^2 f(\mathbf{x})$
- Donner l'expression explicite de $\mathbf{x}_{k+1}$ en fonction de $\mathbf{x}_k$ pour la méthode de Newton à pas constant
- Qu'en déduit-on?

EXEMPLE NON QUADRATIQUE



EXEMPLE NON QUADRATIQUE



Malgré ses nombreux avantages, la méthode de Newton souffre de 2 inconvénients:

- il faut calculer et inverser la matrice Hessienne, ce qui devient très coûteux en grande dimension
- la Hessienne n'est pas toujours définie positive, il n'est donc pas toujours possible de calculer son inverse

→ Méthodes de quasi-Newton

On remplace la matrice Hessienne par une approximation

$$\mathbf{B}_k \approx \nabla^2 f(\mathbf{x}_k)$$

de telle sorte que les itérations deviennent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{B}_k^{-1} \nabla f(\mathbf{x}_k)$$

- On cherche à estimer récursivement les matrices B_k (ou H_k) telles que

$$B_k \approx \nabla^2 f(\mathbf{x}^{(k)}) \quad \text{ou} \quad H_k \approx \left[\nabla^2 f(\mathbf{x}^{(k)}) \right]^{-1}$$

- Une règle de mise à jour $\rightarrow$ une méthode de quasi-Newton

Quelques exemples

- Broyden-Fletcher-Goldfarb-Shanno (BFGS): la plus populaire
- L-BFGS: BFGS à faible coût mémoire
- Symmetric rank-one (SR1)
- Davidon-Fletcher-Powell (DFP)

...

- A l'itération k ,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{H}_k \nabla f(\mathbf{x}_k)$$

- On pose

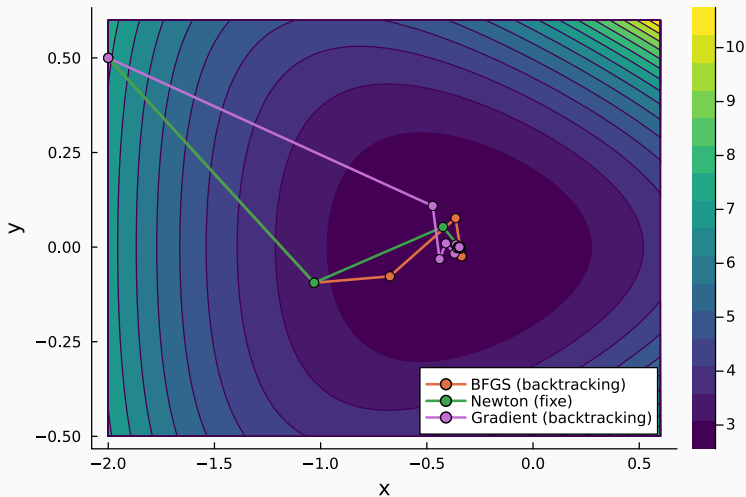
$$\mathbf{s}_k = -\alpha_k \mathbf{H}_k \nabla f(\mathbf{x}_k), \quad \mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k), \quad \rho_k = \frac{1}{\mathbf{s}_k^\top \mathbf{y}_k}$$

Mise à jour de $\mathbf{H}_k$ dans BFGS:

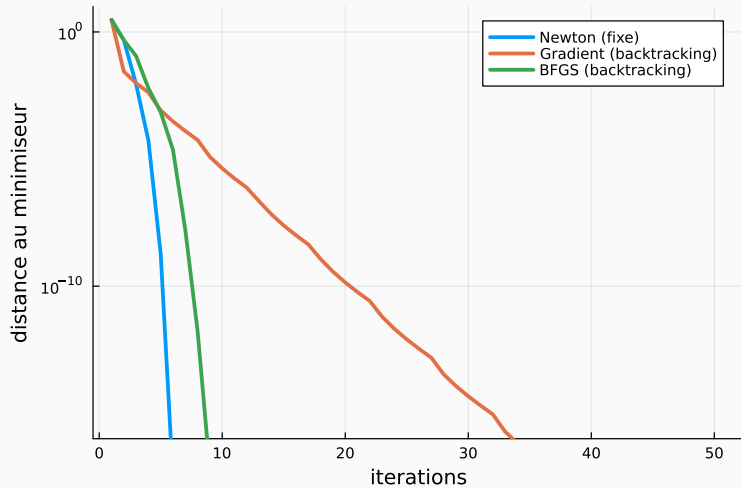
$$\mathbf{H}_{k+1} = \left(\mathbf{I}_n - \rho_k \mathbf{s}_k \mathbf{y}_k^\top \right) \mathbf{H}_k \left(\mathbf{I}_n - \rho_k \mathbf{y}_k \mathbf{s}_k^\top \right) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

- le choix du pas α_k se fait par une technique de recherche de ligne (backtracking, par exemple)

EXEMPLE NON QUADRATIQUE ($H_0 = [\nabla^2 f(x_0)]^{-1}$)



EXEMPLE NON QUADRATIQUE ($H_0 = [\nabla^2 f(x_0)]^{-1}$)



- Forme générale

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) := \frac{1}{2} \sum_{j=1}^m r_j(\mathbf{x})^2, \quad r_j : \mathbb{R}^n \rightarrow \mathbb{R}$$

- Forme générale

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) := \frac{1}{2} \sum_{j=1}^m r_j(\mathbf{x})^2, \quad r_j : \mathbb{R}^n \rightarrow \mathbb{R}$$

- Si on pose

$$R(\mathbf{x}) = (r_1(\mathbf{x}), r_2(\mathbf{x}), \dots, r_m(\mathbf{x}))^\top$$

On peut réécrire le problème de la manière suivante

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) := \frac{1}{2} \|R(\mathbf{x})\|_2^2, \quad R(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

- **Remarque** si $R(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$, on retrouve un problème de moindres carrés linéaires

La forme particulière de $f(\mathbf{x})$ peut être exploitée pour construire des algorithmes efficaces.

Outil clé: matrice Jacobienne de $R : \mathbb{R}^n \rightarrow \mathbb{R}^m$

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial r_1}{\partial x_1}(\mathbf{x}) & \dots & \frac{\partial r_1}{\partial x_N}(\mathbf{x}) \\ \frac{\partial r_2}{\partial x_1}(\mathbf{x}) & \dots & \frac{\partial r_2}{\partial x_N}(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial r_M}{\partial x_1}(\mathbf{x}) & \dots & \frac{\partial r_M}{\partial x_N}(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \nabla r_1(\mathbf{x})^\top \\ \nabla r_2(\mathbf{x})^\top \\ \vdots \\ \nabla r_M(\mathbf{x})^\top \end{bmatrix} \in \mathbb{R}^{m \times n}$$

L'utilisation de la matrice Jacobienne permet d'exprimer simplement le gradient et le Hessien de $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{R}(\mathbf{x})\|_2^2$.

Gradient de f

$$\nabla f(\mathbf{x}) = \sum_{j=1}^m r_j(\mathbf{x}) \nabla r_j(\mathbf{x}) = \mathbf{J}(\mathbf{x})^\top \mathbf{R}(\mathbf{x})$$

Hessienne de f

$$\begin{aligned} \nabla^2 f(\mathbf{x}) &= \sum_{j=1}^m \nabla r_j(\mathbf{x}) \nabla r_j(\mathbf{x})^\top + \sum_{j=1}^m r_j(\mathbf{x}) \nabla^2 r_j(\mathbf{x}) \\ &= \mathbf{J}(\mathbf{x})^\top \mathbf{J}(\mathbf{x}) + \sum_{j=1}^m r_j(\mathbf{x}) \nabla^2 r_j(\mathbf{x}) \end{aligned}$$

Dans de nombreuses applications:

- Calculer le Jacobien $J(\mathbf{x})$ est **peu coûteux** (ou facile à obtenir)
- On en déduit directement le gradient $\nabla f(\mathbf{x}) = J(\mathbf{x})^\top \mathbf{R}(\mathbf{x})$
- Dans le Hessien, bien souvent le second terme peut être **négligé** (notamment près de la solution) de telle sorte que

$$\nabla^2 f(\mathbf{x}) \approx J(\mathbf{x})^\top J(\mathbf{x})$$

c'est à dire que la Hessienne se calcule **sans dérivée seconde**.

→ on exploite ces propriétés pour construire des algorithmes

- méthode de **Gauss-Newton**
- méthode de **Levenberg-Marquardt**

Méthode de Gauss-Newton

c'est une méthode de quasi-Newton avec $\mathbf{B}_k = \mathbf{J}(\mathbf{x}_k)^\top \mathbf{J}(\mathbf{x}_k)$

A l'itération k , ($\alpha_k = 1$)

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left[\mathbf{J}(\mathbf{x}_k)^\top \mathbf{J}(\mathbf{x}_k) \right]^{-1} \nabla f(\mathbf{x}_k)$$

Méthode de Levenberg-Marquardt

peut aussi s'interpréter comme une méthode de quasi-Newton

A l'itération k , ($\alpha_k = 1$), on pose $\mathbf{B}_k = \mathbf{J}(\mathbf{x}_k)^\top \mathbf{J}(\mathbf{x}_k)$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{B}_k + \lambda_k \text{diag}(\mathbf{B}_k)]^{-1} \nabla f(\mathbf{x}_k)$$

où λ_k est un paramètre de régularisation.

Pour un problème donné, le choix d'un algorithme peut se faire en fonction de:

- **sa vitesse de convergence**: elle indique la vitesse à laquelle les itérées se rapprochent de leur limite, mesurée par la distance

$$\varepsilon_k = \|\mathbf{x}_k - \mathbf{x}_\star\|_2 \text{ où } \mathbf{x}_\star \text{ est un minimiseur local}$$

La vitesse de convergence est *asymptotique*, au sens où elle apparaît pour $k \rightarrow \infty$

- **sa complexité**: elle mesure le coût calculatoire associé à un algorithme (en général, une itération de celui-ci)

Distance à un minimiseur local

$$\varepsilon_k = \|\mathbf{x}_k - \mathbf{x}_\star\|_2 \text{ où } \mathbf{x}^\star \text{ est un minimiseur local}$$

- Convergence linéaire:

$$\frac{\varepsilon_{k+1}}{\varepsilon_k} \xrightarrow[k \rightarrow \infty]{} \mu, \quad 0 < \mu < 1$$

- Convergence super-linéaire:

$$\frac{\varepsilon_{k+1}}{\varepsilon_k} \xrightarrow[k \rightarrow \infty]{} 0$$

- Convergence quadratique:

$$\frac{\varepsilon_{k+1}}{\varepsilon_k^2} \xrightarrow[k \rightarrow \infty]{} \mu, \quad 0 < \mu < 1$$

- La complexité se mesure en flops = nombre d'opérations élémentaires (addition, multiplication)
- Complexité des opérations de base
 - produit scalaire $\mathbf{a}^\top \mathbf{b}$ avec $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ $\mathcal{O}(n)$ flops
 - produit matrice-vecteur $\mathbf{A}\mathbf{b}$ avec $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^n$ $\mathcal{O}(mn)$ flops
 - produit matriciel $\mathbf{A}\mathbf{b}$ avec $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^{n \times p}$ $\mathcal{O}(mnp)$ flops
 - inversion matricielle $\mathbf{A}^{-1}$ avec $\mathbf{A} \in \mathbb{R}^{n \times n}$ $\mathcal{O}(n^3)$ flops
- La complexité globale d'une suite d'opérations peut parfois être ardue à calculer, ou ne pas être suffisante pour caractériser le coût computationnel (problèmes liés à l'empreinte mémoire / swap, notamment)

$$\min_{x \in \mathbb{R}^n} f(x)$$

■ méthode du gradient à pas constant

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

- Complexité: $\mathcal{O}(n)$ par itération
- Vitesse de convergence linéaire

■ méthode de Newton

$$x_{k+1} = x_k - \alpha_k \left[\nabla^2 f(x_k) \right]^{-1} \nabla f(x_k)$$

- Complexité: $\mathcal{O}(n^3)$ par itération
- Vitesse de convergence quadratique

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page the extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.